

Universidad Tecnológica Nacional

Facultad Regional Rosario

Ingeniería en Sistemas de Información

SOPORTE A LA GESTIÓN DE DATOS CON PROGRAMACIÓN VISUAL

Informe

Serverless

Qué es, cómo funciona por dentro, cuánto cuesta y cuándo conviene

Profesor/a

(completar)

Alumnos

Nombre y Apellido	Correo electrónico	Legajo
Casermeiro, Gonzalo	(completar correo)	52674
(completar)	(completar)	(completar)
(completar)	(completar)	(completar)
(completar)	(completar)	(completar)

Fecha de entrega: 09/10/2026

Índice

1. Introducción
2. Qué es serverless
3. La escalera de abstracción
4. Cómo funciona por dentro
5. Economía
6. Serverless en la capa de datos
7. Cuándo conviene y cuándo no
8. Limitaciones y seguridad
9. Tendencias en 2026
10. Conclusiones
11. Referencias bibliográficas
12. Anexo: glosario

1. Introducción

Este informe acompaña la exposición sobre *serverless* presentada en la materia. Su objetivo es definir el modelo con precisión, explicar los mecanismos técnicos que lo hacen posible, analizar su economía y delimitar los casos en los que conviene adoptarlo y los casos en los que no.

El tema no es marginal. AWS Lambda, el servicio de funciones de Amazon, ejecuta más de quince billones (10^{12}) de invocaciones por mes, y según el relevamiento de Datadog de 2025 el 65 % de los clientes de AWS utiliza Lambda, el 70 % de los de Google Cloud utiliza Cloud Run y el 56 % de los de Azure utiliza App Service. Se trata de un modelo de ejecución en producción en la mayoría de las organizaciones que operan en la nube.

El trabajo se organiza en el mismo orden que la exposición: qué es (secciones 2 y 3), cómo funciona por dentro (sección 4), economía y datos (secciones 5 y 6), y límites y tendencias (secciones 7 a 9).

2. Qué es serverless

2.1 Definición

Se adopta la definición de la Cloud Native Computing Foundation (CNCF):

Serverless computing refiere al concepto de construir y correr aplicaciones que no requieren gestión de servidores. Describe un modelo donde las aplicaciones, empaquetadas como una o más funciones, se suben a una plataforma y luego se ejecutan, escalan y facturan en respuesta a la demanda exacta del momento. — CNCF, Serverless Whitepaper v1.0.

El nombre induce a error: los servidores existen, están en un centro de datos y alguien los mantiene. Lo que cambia es que ese alguien es el proveedor y no el desarrollador. La situación es análoga a la de *wireless*: los cables siguen existiendo, pero dejan de ser problema del usuario.

2.2 Las tres propiedades

De la definición se desprenden tres propiedades, que se usan en este trabajo como criterio para decidir si un servicio es serverless o no (Tabla 1).

Tabla 1. Propiedades que definen el modelo serverless

Propiedad	Significado
Sin gestión de infraestructura	No se provisionan máquinas, no se parchea el sistema operativo, no se dimensiona capacidad.
Escalado automático hasta cero	La plataforma levanta las instancias necesarias según la demanda y no deja ninguna corriendo si no hay pedidos.
Facturación por uso real	Se paga por ejecución, no por tiempo de servidor encendido.

Fuente: elaboración propia a partir de CNCF.

Si falta cualquiera de las tres, el servicio no es serverless. Un clúster de Kubernetes administrado por el proveedor cumple la primera propiedad sólo a medias y no cumple la tercera: el *pod* tiene costo aunque nadie lo invoque.

2.3 Qué no es serverless

- **No es "sin operaciones"**. La seguridad, los permisos, la observabilidad, el versionado y el despliegue siguen siendo responsabilidad del equipo.
- **No es lo mismo que PaaS**. La diferencia está en la granularidad del escalado y de la facturación: un PaaS cobra por instancia encendida; una plataforma serverless, por ejecución.
- **No es lo mismo que microservicios**. Son conceptos ortogonales: un monolito puede desplegarse como una sola función, y doscientos microservicios pueden correr en máquinas virtuales.

3. La escalera de abstracción

Serverless es el último escalón de una evolución de más de veinte años en la que, en cada paso, el desarrollador deja de ocuparse de una capa más de la infraestructura. La Tabla 2 resume esa progresión.

Tabla 2. Responsabilidades del desarrollador según el modelo de cómputo

Capa	Bare metal	IaaS (EC2)	PaaS (Heroku)	Contenedores (K8s)	FaaS (Lambda)
Código y dependencias	Usuario	Usuario	Usuario	Usuario	Usuario
Runtime	Usuario	Usuario	Proveedor	Usuario	Proveedor
Sistema operativo	Usuario	Usuario	Proveedor	Proveedor	Proveedor
Virtualización y hardware	Usuario	Proveedor	Proveedor	Proveedor	Proveedor
Escalado y capacidad	Usuario	Usuario	Usuario (config.)	Usuario (config.)	Proveedor
Unidad de despliegue	servidor	máquina virtual	aplicación	contenedor	función
Unidad de facturación	compra (CAPEX)	hora	hora	hora (nodo)	milisegundo

Fuente: elaboración propia.

La última fila es la que marca la ruptura: es la primera vez que el costo de cómputo puede ser exactamente cero cuando nadie usa el sistema.

3.1 FaaS y BaaS

La CNCF define serverless como un paraguas sobre dos modelos que conviene separar, porque en la industria suele usarse "serverless" como sinónimo del primero:

- **FaaS (Function as a Service):** código propio, empaquetado en funciones pequeñas que se ejecutan al dispararse un evento o un pedido HTTP. Ejemplos: AWS Lambda, Azure Functions, Google Cloud Run Functions, Cloudflare Workers.
- **BaaS (Backend as a Service):** servicios de terceros consumidos por API que reemplazan funcionalidad que antes se programaba: bases de datos (Firestore, DynamoDB), autenticación (Auth0, Firebase Auth), almacenamiento (S3), pagos (Stripe), búsqueda (Algolia).

En una aplicación serverless madura, la mayor parte del sistema es BaaS y las funciones propias actúan como pegamento entre esos servicios. Se escribe menos código, pero se componen más servicios: la complejidad se desplaza del código a la integración y a la configuración.

4. Cómo funciona por dentro

4.1 Modelo de invocación

Una función serverless no está corriendo a la espera de pedidos: es **invocada** cuando ocurre un evento. La Tabla 3 muestra los tipos de invocación habituales.

Tabla 3. Tipos de invocación

Tipo	Ejemplo	Comportamiento
Síncrona	pedido HTTP a través de un API Gateway	el cliente espera la respuesta
Asíncrona	subida de un archivo a S3	la plataforma encola el evento y reintenta si falla
Por <i>stream</i> o cola	Kinesis, Kafka, SQS	la plataforma lee lotes y los entrega a la función
Programada	<i>cron</i>	la invocación se dispara por tiempo

Fuente: elaboración propia a partir de CNCF y documentación de AWS.

4.2 Ciclo de vida del entorno de ejecución

Cada función corre dentro de un **entorno de ejecución** aislado. Su ciclo de vida tiene tres fases:

1. **INIT.** Sólo ocurre cuando no hay un entorno disponible. La plataforma crea una máquina virtual mínima, descarga el código, arranca el *runtime* y ejecuta el código de inicialización (importaciones, conexión a la base de datos, carga de configuración).
2. **INVOKE.** Se ejecuta la función propiamente dicha con los datos del evento.
3. **FREEZE.** Al terminar, el entorno no se destruye: se congela, con su memoria intacta. Si llega otro pedido, se reutiliza y se omite toda la fase INIT. Si pasan entre cinco y quince minutos sin pedidos, la plataforma lo apaga.

De este ciclo se derivan dos consecuencias de diseño:

- **Las funciones son *stateless* por contrato.** Aunque la memoria sobreviva entre invocaciones del mismo entorno, no hay garantía de que dos pedidos lleguen al mismo entorno. Todo estado de negocio debe guardarse fuera de la función, en una base de datos o un caché.

- **La concurrencia se paga en entornos.** En Lambda cada entorno atiende un solo pedido a la vez. Por la Ley de Little, la cantidad de entornos simultáneos es el producto entre la tasa de pedidos y su duración media: con 100 pedidos por segundo de 0,2 s se necesitan 20 entornos; si cada pedido tarda 2 s, se necesitan 200. El límite por defecto de una cuenta de AWS es de 1.000 ejecuciones simultáneas por región.

4.3 Cold start

Se denomina **cold start** a la latencia adicional de una invocación que debe pasar por la fase INIT porque no había un entorno disponible. Es el precio inevitable de escalar a cero: si no hay nada corriendo, algo hay que arrancar al llegar el primer pedido.

Tabla 4. Cold start típico según runtime (órdenes de magnitud reportados en 2026)

Runtime	Cold start típico
Go / Rust	< 100 ms
Python	200–400 ms
Java con SnapStart	90–140 ms
Java sin optimizar	segundos
Penalidad adicional si la función está en una red privada (VPC)	+200–500 ms

Fuente: fuentes secundarias; no son mediciones propias.

Como muestra la Tabla 4, el lenguaje es el factor dominante. Las mitigaciones disponibles son:

- **Provisioned Concurrency:** mantiene entornos precalentados. Elimina el cold start, pero esos entornos se pagan por hora, con lo que se resigna parte del pago por uso.
- **SnapStart:** en lugar de arrancar desde cero, restaura una instantánea de memoria y disco de un entorno ya inicializado.
- **Paquetes livianos:** menos dependencias implican menos para descargar y cargar.

4.4 Aislamiento: microVMs e *isolates*

El proveedor enfrenta un problema difícil: ejecutar código de miles de clientes en la misma máquina física, sin que uno pueda acceder al otro, y con arranques de milisegundos. Los contenedores no alcanzan, porque comparten el núcleo del sistema operativo; las máquinas virtuales tradicionales son demasiado lentas. Hay dos respuestas en uso.

Firecracker (AWS). Es un monitor de máquinas virtuales de código abierto, escrito en Rust, que se apoya en KVM. Arranca una microVM con Linux en menos de 125 ms con unos 5 MiB de memoria adicional, porque elimina todo el hardware emulado que una función no necesita (BIOS, bus PCI, video, USB). El aislamiento es por virtualización de hardware, reforzado con *namespaces*, *chroot* y filtrado de llamadas al sistema (*seccomp*). Es la base de Lambda y de Fargate.

Isolates de V8 (Cloudflare Workers). En lugar de una máquina virtual por función, un único proceso del motor de JavaScript de Chrome aloja miles de contextos aislados entre sí. Un *isolate* arranca unas cien veces más rápido que un proceso de Node.js, con lo que el cold start

prácticamente desaparece. La contrapartida es que el aislamiento es por software y sólo se puede ejecutar JavaScript, TypeScript o WebAssembly.

Tabla 5. Comparación de los dos modelos de aislamiento

	Firecracker (Lambda)	Isolates V8 (Workers)
Unidad de aislamiento	microVM	contexto del motor JS
Tipo de aislamiento	hardware	software
Arranque	~100 ms	microsegundos
Memoria por instancia	MB	KB
Qué puede ejecutar	cualquier binario Linux	JavaScript / WebAssembly

Fuente: elaboración propia a partir de la documentación de Firecracker y Cloudflare.

5. Economía

5.1 Modelo de facturación

AWS Lambda factura en dos componentes (Tabla 6): una tarifa por cantidad de pedidos y otra por duración, medida en **GB-segundo** (memoria configurada multiplicada por el tiempo de ejecución), con granularidad de un milisegundo.

Tabla 6. Precios de AWS Lambda (región US East, x86)

Concepto	Precio
Capa gratuita mensual	1.000.000 de pedidos y 400.000 GB-s
Pedidos	USD 0,20 por millón
Duración	USD 0,0000166667 por GB-s
Duración en ARM (Graviton)	USD 0,0000133334 por GB-s (~20 % menos)
Granularidad	1 ms

Fuente: AWS Lambda Pricing.

La CPU se asigna en proporción a la memoria: 1.769 MB equivalen a una vCPU completa. Por eso, aumentar la memoria puede no aumentar el costo: si se duplica la memoria y la función termina en la mitad del tiempo, el gasto es el mismo, con la mitad de latencia.

5.2 Caso de baja utilización

Se calcula el costo mensual de una API con 5.000.000 de pedidos por mes, 200 ms de duración media y 512 MB de memoria:

$$\begin{aligned} \text{Pedidos: } & 5.000.000 - 1.000.000 \text{ (capa gratuita)} = 4.000.000 \\ & 4 \times \text{USD } 0,20 = \text{USD } 0,80 \end{aligned}$$

Cómputo:	$5.000.000 \times 0,2 \text{ s} \times 0,5 \text{ GB}$	= 500.000 GB-s
	$500.000 - 400.000 \text{ (capa gratuita)}$	= 100.000 GB-s
	$100.000 \times \text{USD } 0,0000166667$	= USD 1,67

Total $\approx$ USD 2,47 por mes

Una instancia EC2 `t3.small` encendida todo el mes cuesta alrededor de USD 15. Para esta carga, serverless resulta unas seis veces más barato.

5.3 El punto de cruce

El caso anterior es favorable porque la API está ociosa casi todo el tiempo. Si la función corriera sin interrupción, la comparación cambia. Normalizando el precio por vCPU-hora:

Lambda:	$\text{USD } 0,0000166667 \times 3.600 \text{ s} \times 1,769 \text{ GB}$	$\approx \text{USD } 0,106 \text{ por vCPU-hora}$
EC2 <code>c7g.large</code> (2 vCPU,	$\text{USD } 0,0725 \text{ por hora}$	$\approx \text{USD } 0,036 \text{ por vCPU-hora}$

El servidor cuesta lo mismo esté ocupado o no; Lambda arranca en cero pero cada hora de cómputo cuesta unas tres veces más. Las dos curvas de costo se cruzan en $0,036 / 0,106 \approx 34 \%$ **de utilización**. Por debajo de ese valor conviene serverless, y por amplio margen; por encima, una instancia dedicada.

La analogía que resume esta relación es la del auto: tener un servidor es tener auto propio, que se paga aunque esté estacionado; serverless es tomar un taxi, que no cuesta nada si no se viaja pero tiene un kilómetro más caro. Para quien viaja ocho horas por día, conviene el auto.

5.4 Costos que no aparecen en la factura de cómputo

- **Observabilidad:** el registro y el trazado de miles de ejecuciones efímeras puede costar más que el cómputo mismo.
- **Orquestación:** en sistemas con muchas funciones encadenadas, los servicios que las coordinan cobran por transición de estado.
- **Transferencia de datos:** mover datos entre funciones, por red o a través de almacenamiento intermedio, se paga y puede ser el rubro principal.

6. Serverless en la capa de datos

6.1 Bases de datos serverless

El modelo se extendió a la persistencia: capacidad elástica, escalado a cero y facturación por consumo. Para que una base de datos sea serverless tiene que poder apagar el cómputo sin perder los datos, y la técnica que lo permite es **desacoplar cómputo de almacenamiento**: los datos viven en almacenamiento persistente y los nodos que procesan consultas se encienden y apagan según la demanda (Tabla 7).

Tabla 7. Bases de datos serverless

Producto	Modelo	Funcionamiento
Neon	PostgreSQL	Los datos residen en almacenamiento de objetos; el cómputo escala a cero sin actividad y se levanta en segundos ante una nueva consulta.
Aurora Serverless v2	MySQL / PostgreSQL	Un proceso de escalado ajusta la capacidad según la demanda; se factura lo consumido.
DynamoDB (on-demand)	NoSQL clave-valor	Escalado transparente, sin provisionar capacidad de lectura ni escritura.
Firestore, Supabase	BaaS de datos	Base de datos, autenticación y API expuestas como servicio.

Fuente: elaboración propia a partir de documentación de los proveedores.

6.2 El problema de las conexiones

Las funciones serverless y las bases relacionales tradicionales chocan. Un servidor PostgreSQL admite cientos de conexiones simultáneas, no miles. Si quinientas funciones se ejecutan a la vez y cada una abre su propia conexión, la base agota sus conexiones y deja de responder.

Hay dos soluciones habituales:

- **Un pooler de conexiones** entre las funciones y la base (RDS Proxy, PgBouncer), que reutiliza un número reducido de conexiones para muchas funciones.
- **Bases de datos consultables por HTTP** (DynamoDB, el *driver* serverless de Neon), que no requieren mantener conexiones abiertas.

7. Cuándo conviene y cuándo no

Según la CNCF, serverless encaja cuando la carga es:

- **Espigada o impredecible**, con períodos ociosos largos.
- **Disparada por eventos** y paralelizable en unidades independientes.
- **Sin estado** entre pedidos.

Ejemplos típicos: generar miniaturas al subir una imagen, reaccionar a cambios en una base de datos, procesar mensajes de sensores, tareas programadas y APIs con tráfico variable. iRobot, fabricante de la aspiradora Roomba, procesa más de veinte millones de eventos diarios con AWS IoT y Lambda y reporta un 30 % menos de costo que con servidores tradicionales.

No encaja cuando:

- **La carga es constante**: por encima de ~34 % de utilización una instancia dedicada es más barata (sección 5.3).
- **La latencia es crítica**: el cold start no es aceptable.
- **Hace falta estado o conexiones persistentes**.
- **El sistema es una cadena larga de funciones muy acopladas**: el costo de comunicación entre ellas domina.

7.1 El caso Prime Video

En 2023 el equipo de Prime Video publicó que había migrado un sistema de monitoreo de calidad de serverless a una arquitectura monolítica sobre contenedores (Amazon ECS), con una **reducción de costos del 90 %**.

El sistema analizaba la calidad de cada transmisión vista por los usuarios y estaba construido con funciones Lambda coordinadas por Step Functions. Por cada segundo de video se producían varias transiciones de estado, y los datos debían pasar de una etapa a otra por red o mediante almacenamiento intermedio. El costo dominante no era procesar, sino mover datos. La nueva versión ejecuta el conversor de medios y el detector de defectos en un mismo proceso, con los datos en memoria.

La lectura correcta no es que serverless sea un mal modelo, sino que fue elegido para un problema que no le correspondía. Cuando comunicar cuesta más que computar, los componentes deben agruparse, no separarse.

8. Limitaciones y seguridad

8.1 Limitaciones

- **Dependencia del proveedor (*vendor lock-in*).** El acoplamiento no está en la función, que es código portable, sino en todo lo que la rodea: el modelo de eventos, los permisos, la base de datos propietaria y la orquestación. Los contenedores serverless (Cloud Run, Fargate) y los marcos abiertos como Knative reducen este problema.
- **Límites de ejecución.** Lambda corta a los 15 minutos; las tareas más largas deben dividirse, lo que agrega orquestación.
- **Pruebas y depuración.** El entorno local no reproduce fielmente la nube y la naturaleza efímera de las instancias dificulta el monitoreo tradicional.

8.2 Seguridad

El proveedor es responsable de la red, los servidores y el sistema operativo; el desarrollador, del código, la lógica y la configuración. Serverless elimina el parcheo del sistema operativo, pero amplía la responsabilidad sobre permisos y validación de entradas. Entre los riesgos propios que identifica OWASP se destacan:

- **Inyección por eventos:** una función puede dispararse desde una cola, un archivo subido o un cambio en la base, caminos en los que no hay un *firewall* de aplicación delante.
- **Funciones con permisos excesivos:** roles demasiado amplios por la dificultad de mantener un permiso mínimo para cada función.
- **Fuga de datos entre invocaciones:** datos de un usuario guardados en una variable global pueden quedar visibles para la siguiente invocación del mismo entorno.

9. Tendencias en 2026

- **AWS Lambda Managed Instances:** funciones Lambda sobre servidores EC2 dedicados, orientadas por el propio proveedor a "cargas estables o predecibles". Es el reconocimiento explícito del punto de cruce.
- **Lambda MicroVMs** (anunciadas en julio de 2026): entornos con estado y sesiones de hasta ocho horas, que rompen dos supuestos originales del modelo (efímero y sin estado).
- **Inferencia de IA en el edge:** plataformas como Cloudflare Workers o Vercel ejecutan funciones en cientos de ubicaciones cercanas al usuario.
- **Convergencia con contenedores:** el 66 % de las organizaciones que usan funciones serverless también usan orquestación de contenedores (Datadog, 2025). El futuro es híbrido: cada carga se ubica donde le conviene.

10. Conclusiones

- Serverless no significa ausencia de servidores, sino ausencia de su administración: se define por tres propiedades simultáneas (sin gestión de infraestructura, escalado a cero y pago por uso).
- Su funcionamiento se apoya en entornos de ejecución efímeros y fuertemente aislados — microVMs o *isolates*—, de los que derivan tanto sus ventajas como sus restricciones: el cold start y la ausencia de estado.
- Económicamente, es muy superior en cargas espigadas e intermitentes y pierde frente a un servidor dedicado cuando la utilización supera aproximadamente un tercio del tiempo.
- En la capa de datos, la clave es desacoplar cómputo de almacenamiento, y el principal obstáculo práctico es la gestión de conexiones a bases relacionales.
- No es una solución universal sino un compromiso: se elige porque encaja con el problema, no porque sea tendencia. El caso Prime Video y la evolución de las propias plataformas en 2026 lo confirman.

11. Referencias bibliográficas

Amazon Web Services. (2026). *AWS Lambda Pricing*. <https://aws.amazon.com/lambda/pricing/>

Amazon Web Services. (2026). *Introducing AWS Lambda Managed Instances*. AWS News Blog. <https://aws.amazon.com/blogs/aws/introducing-aws-lambda-managed-instances-serverless-simplicity-with-ec2-flexibility/>

Amazon Web Services. (2026, 10 de julio). *Announcing Lambda MicroVMs*. AWS Compute Blog. <https://aws.amazon.com/blogs/compute/announcing-lambda-microvms-serverless-compute-environments-with-vm-level-isolation-and-near-instant-startup/>

Amazon Web Services. (s.f.). *Aurora Serverless v2: how it works*. <https://docs.aws.amazon.com/AuroraServerless/latest/AuroraUserGuide/aurora-serverless-v2-how-it-works.html>

Cloud Native Computing Foundation. (2018). *CNCF Serverless Whitepaper v1.0*. <https://github.com/cncf/wg-serverless/blob/master/whitepapers/serverless-overview/README.md>

Cloudflare. (s.f.). *How Workers works*. <https://developers.cloudflare.com/workers/reference/how-workers-works/>

Datadog. (2025). *State of Containers and Serverless 2025*. <https://www.datadoghq.com/state-of-containers-and-serverless/>

Firecracker. (s.f.). *Firecracker microVMs*. <https://firecracker-microvm.github.io/>

Jonas, E., Schleier-Smith, J., Sreekanti, V. et al. (2019). *Cloud Programming Simplified: A Berkeley View on Serverless Computing*. arXiv:1902.03383. <https://arxiv.org/abs/1902.03383>

OWASP. (s.f.). *OWASP Serverless Top 10*. <https://owasp.org/www-project-serverless-top-10/>

Roberts, M. (2018). *Serverless Architectures*. martinowler.com. <https://martinfowler.com/articles/serverless.html>

DevClass. (2023, 5 de mayo). *Reduce costs by 90% by moving from microservices to monolith: Amazon internal case study raises eyebrows*. DevClass. <https://www.devclass.com/ci-cd/2023/05/05/reduce-costs-by-90-by-moving-from-microservices-to-monolith-amazon-internal-case-study-raise-s-eyebrows/1621790>

12. Anexo: glosario

Término	Definición
FaaS	<i>Function as a Service</i> . Código propio, ejecutado por eventos, que escala a cero.
BaaS	<i>Backend as a Service</i> . Servicios de backend consumidos como API gestionada.
Cold start	Latencia adicional de una invocación que debe inicializar un entorno nuevo.
Warm start	Invocación que reutiliza un entorno ya inicializado.
Entorno de ejecución	Espacio aislado donde corre la función; se congela y reutiliza entre invocaciones.
Escalar a cero	Que la plataforma no mantenga ninguna instancia activa cuando no hay tráfico.
GB-segundo	Unidad de facturación: memoria asignada multiplicada por tiempo de ejecución.
microVM	Máquina virtual mínima, sin hardware heredado, que arranca en milisegundos.
Isolate	Contexto liviano de V8 que aísla código dentro de un mismo proceso.
Provisioned Concurrency	Entornos precalentados que se pagan por hora para evitar el cold start.
Vendor lock-in	Dependencia de un proveedor que encarece la migración a otro.